Full Stack Development – 5-Mark Answers

Q1. Explain the difference between var, let, and const with examples.

Definition: In JavaScript, var, let, and const are keywords used to declare variables. They differ in scope, redeclaration, and reassignment.

Important Points:

1. var: It is function-scoped and allows redeclaration and reassignment.
2. let: It is block-scoped and allows reassignment but does not allow redeclaration in the same scope.
3. const: It is block-scoped and does not allow redeclaration or reassignment.
4. Scope: var works within a function, whereas let and const work within the nearest block {}.
5. Usage: Use let when a value changes and const when a variable should not be reassigned.

Comparison Table:

Feature	**var**	**let**	**const**
Scope	Function	Block	Block
Redeclaration	Allowed	Not allowed in same scope	Not allowed in same scope
Reassignment	Allowed	Allowed	Not allowed
Initialization	Optional	Optional	Required

Example:

```
var name = "Rishabh";
name = "Rahul"; // Allowed

let age = 20;
age = 21; // Allowed

const country = "India";
// country = "USA"; // Error

console.log(name);
console.log(age);
console.log(country);
```

OUTPUT :

```
Rahul
21
India
```

Q2. How does JavaScript handle user events? Explain with examples.

Definition: An event in JavaScript is an action that occurs in a web page, such as a mouse click, key press, or form submission. JavaScript handles events using event handlers and event listeners.

Important Points:

1. User Action: An event occurs when a user interacts with a web page.
2. Event Handler: A function executes when a particular event occurs.
3. addEventListener(): It attaches an event listener to an HTML element.
4. Common Events: click, mouseover, keydown, change, and submit.
5. Event Object: It provides information about the event, such as the target element.
6. DOM Interaction: JavaScript can change page content or styles in response to events.

Example:

```
<button id="btn">Click Me</button>
<p id="result"></p>

<script>
document.getElementById("btn")
  .addEventListener("click", function() {
    document.getElementById("result")
      .textContent = "Button Clicked!";
  });
</script>
```

Output:

Before clicking: Button Click Me is displayed.

After clicking: `Button Clicked!` appears below the button.

Working:

1. The user clicks the button.
2. The click event occurs.
3. The event listener executes the function.
4. The paragraph content changes using textContent.

Q3. What is the difference between == and === in JavaScript?

Definition: In JavaScript, == and === are comparison operators used to compare two values. The main difference is that == allows type conversion, whereas === compares both value and data type without type conversion.

Important Points:

1. == (Loose Equality): Compares values after converting their types when necessary.
2. === (Strict Equality): Compares both value and data type.
3. Type Conversion: == may automatically convert a string to a number.
4. Strict Comparison: === does not perform automatic type conversion.
5. Best Practice: Use === in most cases to avoid unexpected results.

Comparison Table:

Feature	**==**	**===**
Name	Loose equality	Strict equality
Value comparison	Yes	Yes
Data type comparison	May convert types	Checks types
Type conversion	Possible	No
Example	5 == "5" → true	5 === "5" → false

Example:

```
let a = 10;
let b = "10";

console.log(a == b);
console.log(a === b);

console.log(5 == "5");
console.log(5 === "5");
```

OUTPUT:

```
true
false
true
false
```

Explanation: The == operator treats the number 10 and the string "10" as equal after type conversion. The === operator returns false because their data types are different.

Q4. Explain JavaScript functions and their different types.

Definition: A function in JavaScript is a reusable block of code designed to perform a specific task. It executes when it is called.

Important Points:

1. Function Declaration: A function is defined using the function keyword and a name.
2. Function Expression: A function is stored in a variable.
3. Arrow Function: A shorter way to write functions using the => symbol.
4. Anonymous Function: A function without a name, often used as a callback.
5. Parameters and Arguments: Parameters receive values, while arguments are the actual values passed to a function.
6. Return Statement: The return statement sends a result back to the caller.

Example:

```
// Function Declaration
function add(a, b) {
    return a + b;
}

// Function Expression
const sub = function(a, b) {
    return a - b;
};

// Arrow Function
const mul = (a, b) => a * b;

console.log(add(5, 3));
console.log(sub(5, 3));
console.log(mul(5, 3));
```

OUTPUT:

```
8
2
15
```

Different Types of Functions:

Type	**Description**
Function Declaration	Defined using the function keyword
Function Expression	Assigned to a variable
Arrow Function	Uses the => symbol
Anonymous Function	Has no function name
Callback Function	Passed as an argument to another function

Q5. What is the Document Object Model (DOM)? Explain its importance.

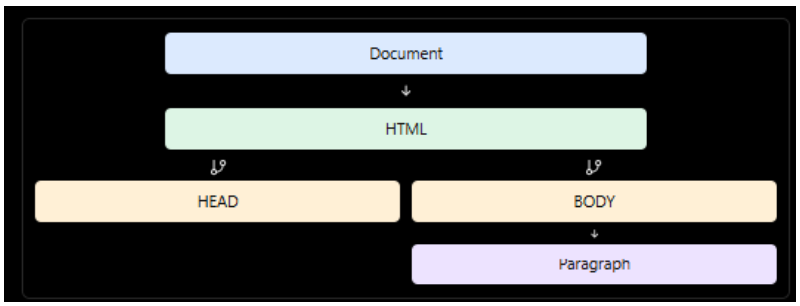
Definition: The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. JavaScript uses the DOM to access and modify web page content, structure, and styles.

Important Points:

1. Tree Structure: The DOM represents HTML elements as a hierarchical tree of nodes.
2. Access Elements: JavaScript can select elements using methods such as getElementById() and querySelector().
3. Change Content: The textContent and innerHTML properties can modify element content.
4. Modify Styles: JavaScript can change CSS styles using the style property.
5. Add or Remove Elements: The DOM allows developers to create, insert, and remove HTML elements.

6. Handle Events: The DOM works with event listeners to respond to user actions.

DOM Tree Diagram:



Example:

```
<p id="demo">Hello World</p>

<script>
document.getElementById("demo")
  .textContent = "Welcome to JavaScript!";
</script>
```

OUTPUT:

```
Welcome to JavaScript!
```

Q6. Explain the Window Object and its commonly used methods.

Definition: The window object is the global object in a web browser. It represents the browser window and provides methods and properties to interact with the browser.

Important Points:

1. alert(): Displays a message in a dialog box.
2. confirm(): Displays a dialog box with OK and Cancel options and returns true or false.
3. prompt(): Accepts input from the user through a dialog box.
4. setTimeout(): Executes a function once after a specified delay.
5. setInterval(): Repeatedly executes a function after a specified interval.
6. open(): Opens a new browser window or tab, subject to browser restrictions.

Example:

```
window.alert("Welcome!");


let result = window.confirm("Do you want to continue?");
console.log(result);


let name = window.prompt("Enter your name:");
console.log(name);


window.setTimeout(function() {
  console.log("Hello!");
}, 2000);
```

Output:

- An alert displays Welcome!.
- A confirmation dialog asks whether to continue.

- A prompt asks for the user's name.
- After 2 seconds, Hello! appears in the console.

Commonly Used Window Methods:

Method	Purpose
alert()	Displays a message
confirm()	Gets OK/Cancel confirmation
prompt()	Takes user input
setTimeout()	Runs code once after a delay
setInterval()	Runs code repeatedly
open()	Opens a window or tab

Q7. What are JavaScript Objects? Explain with an example.

Definition: A JavaScript object is a collection of properties stored as key-value pairs. It is used to represent real-world entities and store related data and functions together.

Important Points:

1. Properties: Objects store data in key-value pairs, such as name: "Rishabh".
2. Methods: Objects can contain functions that perform specific tasks.
3. Object Creation: Objects can be created using object literals {}.
4. Accessing Properties: Properties can be accessed using dot notation or bracket notation.
5. Modifying Objects: Properties can be added, updated, or deleted.
6. Real-World Use: Objects represent students, employees, products, and other entities.

Example:

```
const student = {
    name: "Rishabh",
    age: 20,
    course: "B.Sc. IT",

    display: function() {
        return this.name;
    }
};

console.log(student.name);
console.log(student.age);
console.log(student.display());
```

OUTPUT:

```
Rishabh
20
Rishabh
```

Explanation:

- name, age, and course are properties.
- display() is a method.
- this.name accesses the name property of the current object.

Q8. Explain the different looping statements available in JavaScript.

Definition: Looping statements in JavaScript are used to execute a block of code repeatedly until a specified condition is met or the items in a collection are processed.

Important Types of Loops:

1. for Loop: Used when the number of iterations is known.
2. while Loop: Repeats code while the condition is true.
3. do...while Loop: Executes the code at least once before checking the condition.
4. for...of Loop: Iterates over values of arrays and other iterable objects.
5. for...in Loop: Iterates over enumerable property keys of an object.
6. Loop Control: break terminates a loop, while continue skips the current iteration.

Example:

```
// For loop
for (let i = 1; i <= 3; i++) {
    console.log(i);
}

// While loop
let j = 1;
while (j <= 3) {
    console.log(j);
    j++;
}

// Do-while loop
let k = 1;
do {
    console.log(k);
    k++;
} while (k <= 3);
```

OUTPUT:

```
1
2
3
1
2
3
1
2
3
```

Q9. What is event bubbling and event capturing in JavaScript?

Definition: Event bubbling and event capturing are two phases that determine the order in which event handlers execute when an event occurs on a nested HTML element.

Important Points:

1. Event Bubbling: The event starts at the target element and moves upward to its parent elements.
2. Event Capturing: The event travels from the outer parent elements down toward the target element.
3. Event Target: The element on which the event originally occurs is called the target.
4. addEventListener(): The third parameter controls the phase; true enables capturing, while false or omission uses bubbling.
5. Event Propagation: stopPropagation() stops the event from continuing through the propagation path.
6. Usage: These phases help developers control event handling in nested elements.

Diagram:

Event Capturing (Top to Bottom) → Parent → Child → Target

Event Bubbling (Bottom to Top)

Target → Child → Parent

Example:

```html
<div id="parent">
    <button id="child">Click Me</button>
</div>

<script>
document.getElementById("parent")
    .addEventListener("click", function() {
        console.log("Parent clicked");
    });

document.getElementById("child")
    .addEventListener("click", function() {
        console.log("Child clicked");
    });
</script>
```

OUTPUT:

```
Child clicked
Parent clicked
```

Q10. Explain JavaScript type conversion with suitable examples.

Definition: Type conversion in JavaScript is the process of changing a value from one data type to another, such as converting a string into a number or a number into a string.

Important Points:

1. Implicit Conversion: JavaScript automatically converts data types when required.
2. Explicit Conversion: The programmer manually converts a value using methods such as Number() and String().
3. Number(): Converts a suitable value into a number.
4. String(): Converts a value into a string.
5. Boolean(): Converts a value into true or false.
6. parseInt(): Converts the beginning of a string into an integer when possible.

Example:

```javascript
// Implicit conversion
let a = "5" + 2;
console.log(a);

// Explicit conversion
let b = Number("10");
console.log(b + 5);

let c = String(100);
console.log(c);

let d = Boolean(1);
console.log(d);

let e = parseInt("25px");
console.log(e);
```

OUTPUT:

```
52
15
100
true
25
```

Q11. How does jQuery simplify DOM manipulation compared to JavaScript?

Definition: jQuery is a lightweight JavaScript library that simplifies HTML document manipulation, event handling, animations, and AJAX operations with less code.

Important Points:

1. Easy Selection: jQuery uses CSS-style selectors to select HTML elements.
2. DOM Manipulation: Methods like html(), text(), and append() make it easy to change page content.
3. Event Handling: The on() method handles events such as clicks and key presses.
4. CSS Manipulation: The css() method changes element styles easily.
5. Effects and Animations: Methods like hide(), show(), and fadeIn() create visual effects.
6. Cross-Browser Support: jQuery simplifies many browser differences in supported operations.

Example:

JavaScript:

```
document.getElementById("demo").textContent = "Hello!";
```

jQuery:

```
$("#demo").text("Hello!");
```

Comparison Table:

JavaScript	**jQuery**
Uses native DOM methods	Uses simple library methods
getElementById("demo")	$("#demo")
textContent = "Hello"	.text("Hello")
Event handling uses native listeners	Event handling uses .on()
DOM operations may need more code	Often requires less code

Conclusion: jQuery makes common DOM operations shorter and easier to write, although modern JavaScript also provides many built-in methods for these tasks.

Q12. Explain jQuery selectors with suitable examples.

Definition: jQuery selectors are used to find and select HTML elements based on their ID, class, tag name, attributes, or other conditions.

Important Points:

1. Element Selector: Selects all elements of a particular HTML tag.
2. ID Selector: Selects an element using its unique ID.
3. Class Selector: Selects all elements belonging to a particular class.
4. Universal Selector: Selects all elements using *.
5. Attribute Selector: Selects elements based on an attribute or its value.
6. Group Selector: Selects multiple types of elements together.

Example:

```
<p id="demo">Hello World</p>
```

Welcome

<button>Click Me</button>

<script>

$("p").css("color", "blue");

$("#demo").text("Hello Rishabh");

$(".msg").css("color", "green");

$("*").css("font-family", "Arial");

</script>

Explanation:

- $("p") selects all paragraph elements.
- $("#demo") selects the element with ID demo.
- $(".msg") selects elements with class msg.
- $("*") selects all elements.

Common jQuery Selectors:

Selector	**Purpose**
$("p")	Selects all paragraphs
$("#id")	Selects an element by ID
$(".class")	Selects elements by class
$("*")	Selects all elements
$("[type='text']")	Selects elements with the specified attribute value
$("h1, p")	Selects all h1 and p elements

Q13. What are jQuery effects? Explain any four methods.

Definition: jQuery effects are methods used to show, hide, fade, or animate HTML elements to make web pages more interactive.

Four Important Methods:

1. hide(): Hides the selected element.
2. show(): Displays a hidden element.
3. fadeIn(): Gradually makes an element visible by increasing its opacity.
4. slideUp(): Hides an element with an upward sliding effect.

Example:

Welcome to jQuery!

<button id="hide">Hide</button>

<button id="show">Show</button>

```
<button id="fade">Fade In</button>
<button id="slide">Slide Up</button>
<script>
$("#hide").click(function() {
  $("#demo").hide();
});
$("#show").click(function() {
  $("#demo").show();
});
$("#fade").click(function() {
  $("#demo").fadeIn();
});
$("#slide").click(function() {
  $("#demo").slideUp();
});
</script>
```

Explanation:

- Clicking Hide hides the paragraph.
- Clicking Show displays the paragraph.
- Clicking Fade In gradually displays it.
- Clicking Slide Up hides it with a sliding effect.

Common jQuery Effects:

Method	**Purpose**
hide()	Hides an element
show()	Displays an element
fadeIn()	Gradually displays an element
fadeOut()	Gradually hides an element
slideDown()	Displays an element with a downward slide
slideUp()	Hides an element with an upward slide

Q14. How does jQuery AJAX simplify asynchronous communication?

Definition: AJAX stands for Asynchronous JavaScript and XML. It allows a web page to exchange data with a server and update content without reloading the entire page. jQuery simplifies AJAX requests through methods such as $.ajax(), $.get(), and $.post().

Important Points:

1. Asynchronous Communication: Sends and receives data while the user continues interacting with the page.
2. Less Code: jQuery provides simple methods to send server requests.
3. $.get(): Sends an HTTP GET request to retrieve data.
4. $.post(): Sends an HTTP POST request, commonly used to submit data.
5. Success and Error Handling: Callback functions handle successful responses and request failures.
6. Dynamic Updates: The returned data can be displayed on the page without a full reload.

Example:

```
$.ajax({
  url: "data.txt",
  method: "GET",
  success: function(response) {
    $("#result").text(response);
  },
  error: function() {
    $("#result").text("Error loading data");
  }
});
```

HTML:

```
<p id="result">Loading...</p>
```

Explanation:

1. $.ajax() sends a request to data.txt.
2. The server returns a response.
3. The success callback displays the response inside the paragraph.
4. If the request fails, the error callback displays an error message.

Common jQuery AJAX Methods:

Method	**Purpose**
$.ajax()	Performs a configurable AJAX request
$.get()	Retrieves data using GET
$.post()	Sends data using POST
.load()	Loads server content into an HTML element

Q15. Explain jQuery Traversing methods with examples.

Definition: jQuery traversing is the process of moving through the DOM tree to find parent, child, or sibling elements of a selected HTML element.

Important Points:

1. parent(): Selects the immediate parent of an element.
2. children(): Selects the direct children of an element.
3. find(): Selects matching descendant elements at any depth.
4. siblings(): Selects sibling elements of the selected element.
5. next(): Selects the immediately following sibling.
6. prev(): Selects the immediately preceding sibling.

Example:

```
<div id="parent">
  <p id="first">First</p>
  <p id="second">Second</p>
  <span>Message</span>
</div>


<script>
$("#second").parent().css("border", "1px solid red");
$("#parent").children("p").css("color", "blue");
$("#parent").find("span").css("color", "green");
$("#second").siblings().css("background", "yellow");
</script>
```

Explanation:

- parent() selects the div containing the second paragraph.
- children("p") selects the direct paragraph children.
- find("span") selects the span inside the div.
- siblings() selects the other sibling elements of the second paragraph.

Common jQuery Traversing Methods:

Method	**Purpose**
parent()	Selects the immediate parent
parents()	Selects all ancestor elements
children()	Selects direct child elements
find()	Selects matching descendants
siblings()	Selects sibling elements
next()	Selects the next sibling
prev()	Selects the previous sibling

Q16. Explain the advantages of HTML5 over previous HTML versions.

Definition: HTML5 is the fifth major version of HTML used to structure and present content on web pages. It introduces semantic elements, multimedia support, and improved web application features.

Important Points:

1. Semantic Elements: HTML5 introduces tags like <header>, <footer>, <article>, and <section> to describe page structure clearly.
2. Multimedia Support: The <audio> and <video> tags allow media playback without external plugins.
3. Graphics Support: The <canvas> element supports drawing graphics and animations using JavaScript.
4. Form Improvements: New input types such as email, date, and number provide better input validation.
5. Web Storage: localStorage and sessionStorage allow data to be stored in the browser.
6. Geolocation API: With user permission, web applications can access the device's location through the Geolocation API.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML5 Example</title>
</head>
<body>
  <header>
    <h1>My Website</h1>
  </header>
  <video controls width="300">
    <source src="video.mp4" type="video/mp4">
  </video>
  <input type="email" placeholder="Enter email">
</body>
</html>
```

Feature	Previous HTML versions	HTML5
Page structure	More generic elements	Semantic elements
Audio/video	Often required plugins	Native audio/video tags
Graphics	Limited native support	Canvas and SVG support
Form input types	Fewer built-in types	More input types
Browser storage	Often relied on cookies	Web Storage APIs

Q17. What are semantic elements in HTML5? Explain any five semantic tags.

Definition: Semantic elements are HTML elements that clearly describe the meaning and purpose of their content to browsers and developers.

Five Important Semantic Tags:

1. <header>: Defines introductory content, such as a page title, logo, or header navigation.
2. <nav>: Contains important navigation links to different pages or sections.
3. <main>: Represents the main content of the web page.
4. <section>: Groups related content under a common theme or heading.
5. <footer>: Defines footer information, such as copyright details or contact links.

Example:

```
<!DOCTYPE html>
<html>
<body>
  <header>
    <h1>My Website</h1>
  </header>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
  </nav>
  <main>
    <section id="home">
      <h2>Welcome</h2>
      <p>This is my website.</p>
    </section>
  </main>
  <footer>
    <p>Copyright 2026</p>
  </footer>
</body>
</html>
```

Other Semantic Tags:

Tag	Purpose
<article>	Independent content, such as a blog post
<aside>	Related or supplementary content

<figure>	Groups images, diagrams, or illustrations
<figcaption>	Provides a caption for a figure
<time>	Represents a date or time

Importance: Semantic tags improve code readability, accessibility, and search engine understanding of web page structure.

Q18. Differentiate between and semantic HTML5 elements with examples.

Definition: The element is a generic container used to group HTML elements, whereas semantic HTML5 elements describe the purpose and meaning of the content they contain.

Comparison Table:

Feature		Semantic HTML5 elements
Meaning	Generic container	Meaningful, named purpose
Purpose	Groups content for styling or layout	Defines page structure and content roles
Examples		<header>, <nav>, <main>, <footer>
Readability	Meaning depends on its content and attributes	Purpose is clearer from the tag
Accessibility	No specific semantic meaning by default	Provides useful structural information
SEO	No inherent semantic meaning	Helps search engines understand content structure

Example using :

```
<div class="header">
  <h1>My Website</h1>
</div>
<div class="content">
  <p>Welcome to my website.</p>
</div>
```

Example using semantic elements:

```
<header>
  <h1>My Website</h1>
</header>
<main>
  <p>Welcome to my website.</p>
</main>
```

Explanation:

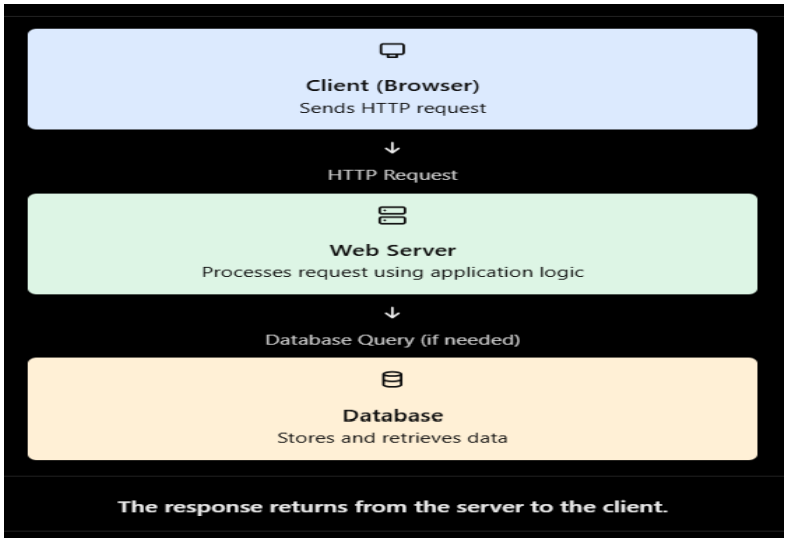
- In the first example, groups the content but does not identify its role.

- In the second example, <header> identifies introductory content and <main> identifies the primary content.
- Both approaches can be styled using CSS, but semantic elements communicate the document structure more clearly.

Q19. Explain the Client-Server Architecture with a neat diagram.

Definition: Client-server architecture is a network model in which a client sends requests to a server, and the server processes those requests and returns responses.

Diagram:



Important Points:

1. Client: The client is usually a web browser that sends requests and displays the results.
2. Server: The server receives requests, executes application logic, and generates responses.
3. Database: Stores application data, such as user details, products, and orders.
4. Communication: The client and server commonly communicate using HTTP or HTTPS.
5. Request and Response: The client sends a request, and the server returns data or a web page.
6. Security: Authentication, authorization, and server-side validation help protect application data.

Example: In an online shopping website, the client requests product details, the server retrieves the products from the database, and the browser displays them.

Q20. Differentiate between Client-Side Scripting and Server-Side Scripting with examples.

Definition: Client-side scripting runs in the user's browser, whereas server-side scripting runs on the web server to process requests and perform backend operations.

Comparison Table:

Feature	Client-Side Scripting	Server-Side Scripting
Execution	User's browser	Web server
Main purpose	UI and interactivity	Business logic and data processing
Languages	JavaScript	PHP, Python, Node.js

Code visibility	Browser-delivered code can be viewed	Server code is generally not sent to the browser
Database access	Usually through a backend/API	Can access databases securely
Speed	Immediate for local UI changes	Requires server communication for processing
Example	Form validation	Login verification

Example of Client-Side Scripting:

```
<button onclick="alert('Hello!')">
  Click Me
</button>
```

Output: A popup displays Hello! when the button is clicked.

Example of Server-Side Scripting (PHP):

```
<?php
$name = "Rishabh";
echo "Welcome " . $name;
?>
```

Output:

Welcome Rishabh

Q21. Explain the purpose of the addEventListener() method with an example. How does it differ from inline event handling?

Definition: The addEventListener() method in JavaScript attaches an event handler to an HTML element. It allows a function to execute when an event, such as a click, occurs.

Important Points:

1. Event Handling: Responds to events such as click, mouseover, and keydown.
2. Separate Code: Keeps JavaScript separate from HTML, improving readability.
3. Multiple Listeners: Allows multiple event listeners for the same event on an element.
4. Event Object: Provides event information to the callback function.
5. Event Removal: A listener can be removed using removeEventListener() when the same function reference is available.
6. Capturing and Bubbling: Its optional third argument can control the event propagation phase.

Example using addEventListener():

```
<button id="btn">Click Me</button>


<script>
document.getElementById("btn")
  .addEventListener("click", function() {
    alert("Button clicked!");
  });
```

</script>

Inline Event Handling:

```
<button onclick="alert('Button clicked!')">
  Click Me
</button>
```

Comparison Table:

Feature	addEventListener()	Inline Event Handling
Location	JavaScript code	HTML attribute
Syntax	element.addEventListener()	onclick="..."
Multiple handlers	Supports multiple listeners	One inline handler per event attribute
Code organization	Separates HTML and JavaScript	Mixes behavior with HTML
Reusability	Easier to manage and remove handlers	Harder to manage for complex behavior

Q22. What is event delegation in JavaScript? Why is it useful?

Definition: Event delegation is a technique in which a single event listener is attached to a parent element to handle events triggered by its child elements. It works by using event bubbling.

Important Points:

1. Single Listener: One parent listener can handle events from multiple child elements.
2. Event Bubbling: Events from child elements bubble up to the parent.
3. Dynamic Elements: It can handle newly added child elements without attaching separate listeners to them.
4. Less Code: Reduces the number of event listeners required.
5. Better Management: Makes event handling easier for lists, menus, and tables.
6. Target Checking: The event.target property helps identify which child element triggered the event.

Example:

```
<ul id="menu">
  <li>Home</li>
  <li>About</li>
  <li>Contact</li>
</ul>
<script>
document.getElementById("menu")
  .addEventListener("click", function(event) {
    if (event.target.tagName === "LI") {
      alert(event.target.textContent);
```

```
    }
  });
</script>
```

Output: Clicking Home, About, or Contact displays the text of the selected item.

Why is it useful?

- Only one listener is required for the entire list.
- Newly added <li> elements can also be handled.
- It reduces repetitive code and simplifies event management.

Q23. Explain different String methods in JavaScript with examples.

Definition: A string in JavaScript is a sequence of characters used to store and manipulate text. JavaScript provides built-in string methods to perform operations such as searching, extracting, and changing text.

Important String Methods:

1. toUpperCase(): Converts all letters into uppercase.
2. toLowerCase(): Converts all letters into lowercase.
3. charAt(): Returns the character at a specified index.
4. slice(): Extracts a portion of a string.
5. includes(): Checks whether a string contains specified text.
6. trim(): Removes whitespace from the beginning and end of a string.

Example:

```
let str = " Hello JavaScript ";

console.log(str.toUpperCase());
console.log(str.toLowerCase());
console.log(str.charAt(1));
console.log(str.slice(1, 6));
console.log(str.includes("Java"));
console.log(str.trim());
```

OUTPUT:

```
 HELLO JAVASCRIPT
 hello javascript
H
Hello
true
Hello JavaScript
```

Other Common String Methods:

Method	**Purpose**
length	Returns the number of characters
replace()	Replaces matching text
indexOf()	Returns the first matching index or -1
concat()	Joins strings
split()	Splits a string into an array

Q24. Explain different Math object methods in JavaScript.

Definition: The JavaScript Math object provides built-in properties and methods for performing mathematical calculations such as rounding, finding powers, and generating random numbers.

Important Math Methods:

1. Math.round(x): Rounds a number to the nearest integer.
2. Math.ceil(x): Rounds a number upward to the next integer.
3. Math.floor(x): Rounds a number downward to the next integer.
4. Math.sqrt(x): Returns the square root of a number.
5. Math.pow(x, y): Returns \(x\) raised to the power \(y\).
6. Math.random(): Generates a pseudo-random number greater than or equal to 0 and less than 1.

```
console.log(Math.round(4.6));
console.log(Math.ceil(4.2));
console.log(Math.floor(4.8));
console.log(Math.sqrt(25));
console.log(Math.pow(2, 3));
console.log(Math.max(10, 20, 30));
console.log(Math.min(10, 20, 30));
```

OUTPUT:

```
5
5
4
5
8
30
10
```

Common Math Methods:

Method	**Purpose**
Math.abs(x)	Returns the absolute value
Math.round(x)	Rounds to the nearest integer
Math.ceil(x)	Rounds upward
Math.floor(x)	Rounds downward
Math.sqrt(x)	Calculates the square root
Math.max()	Returns the largest value
Math.min()	Returns the smallest value
Math.random()	Generates a pseudo-random number

Q25. Explain Date Object methods in JavaScript.

Definition: The JavaScript Date object is used to work with dates and times. It provides methods to retrieve and modify information such as the year, month, day, hours, and minutes.

Important Date Methods:

1. getFullYear(): Returns the four-digit year.
2. getMonth(): Returns the month from 0 to 11, where January is 0.
3. getDate(): Returns the day of the month from 1 to 31.
4. getDay(): Returns the day of the week from 0 (Sunday) to 6 (Saturday).
5. getHours(): Returns the hour from 0 to 23.
6. getMinutes(): Returns the minutes from 0 to 59.

Example:

```
let today = new Date();

console.log(today.getFullYear());
console.log(today.getMonth());
console.log(today.getDate());
console.log(today.getDay());
console.log(today.getHours());
console.log(today.getMinutes());
```

Output: The output depends on the current date and time. For example, if the year is 2026, getFullYear() returns 2026.

Other Common Date Methods:

Method	**Purpose**
getSeconds()	Returns the seconds
getTime()	Returns milliseconds since 1 January 1970 UTC
setFullYear()	Sets the year
setMonth()	Sets the month
toDateString()	Converts the date portion to a readable string
toISOString()	Returns a date-time string in ISO format (UTC)

Example of creating a specific date:

let d = new Date("2026-10-13");

console.log(d.getFullYear());

console.log(d.toDateString());

Output:

2026

Tue Oct 13 2026

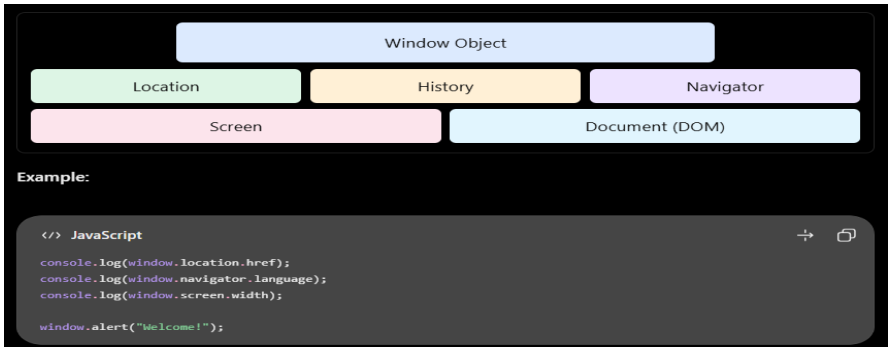
Q26. What is Browser Object Model (BOM)? Explain its commonly used objects.

Definition: The Browser Object Model (BOM) allows JavaScript to interact with the web browser outside the HTML document. Its main global object is window.

Important BOM Objects:

1. Window: Represents the browser window and provides methods such as alert(), confirm(), and setTimeout().
2. Location: Provides information about the current URL and allows navigation using properties such as href.
3. History: Allows navigation through the browser's session history using methods such as back() and forward().
4. Navigator: Provides information about the browser and its environment.
5. Screen: Provides information about the user's screen, such as its width and height.
6. Document: Provides access to the HTML document through the DOM. It is accessible through window.document.

Diagram:



Explanation:

- location.href displays the current URL.
- navigator.language returns the browser's preferred language.
- screen.width returns the screen width in pixels.
- alert() displays a message box.

Importance: BOM features help web applications manage navigation, access browser information, display dialogs, and interact with the browser environment.

Q27. Explain JavaScript Error Handling using try, catch, throw, and finally.

Definition: Error handling in JavaScript is the process of detecting and managing runtime errors so that a program can respond appropriately instead of failing unexpectedly.

Important Keywords:

1. try: Contains code that may produce an error.
2. catch: Handles an error thrown inside the associated try block.
3. throw: Creates and throws a custom error or another value.
4. finally: Executes after the try and catch processing, whether an error occurs or not.
5. Error Object: An Error object can provide details about the error through properties such as message.
6. Benefit: Error handling helps display meaningful messages and makes applications more reliable.

Example:

```
try {
  let age = 15;
  if (age < 18) {
    throw new Error("Not eligible to vote");
  }
  console.log("Eligible to vote");
}
catch (error) {
  console.log(error.message);
```

```
}
finally {
   console.log("Program completed");
}
```

Output:

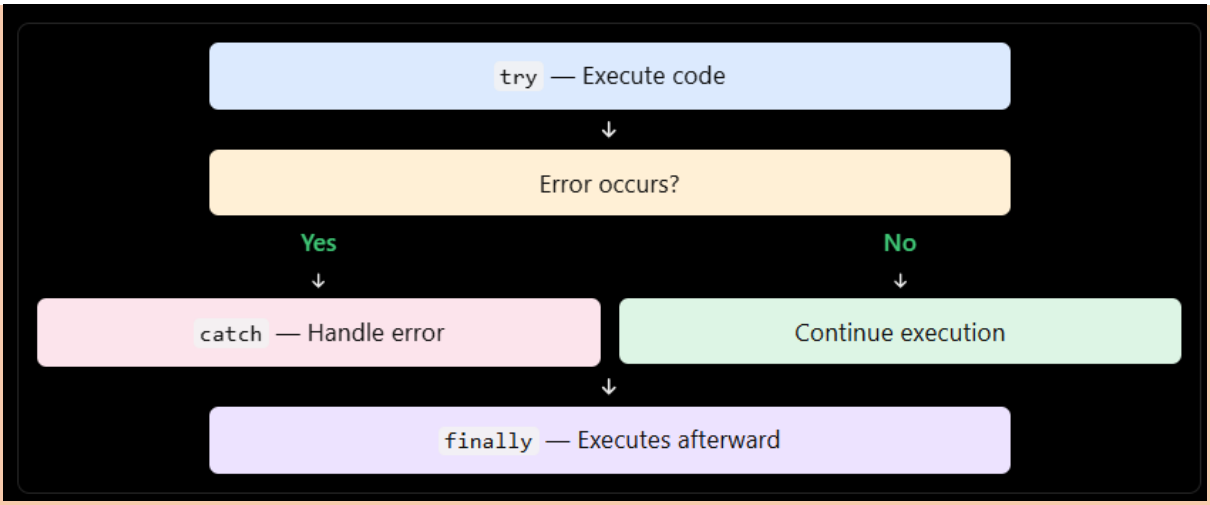
Not eligible to vote

Program completed

Working:

- try executes the code that checks the age.
- throw creates an error when the age is below 18.
- catch handles the error and prints its message.
- finally executes after the error handling.

Flow Diagram:



Q28. What is Strict Mode in JavaScript? Explain its advantages.

Definition: Strict Mode is a feature in JavaScript that helps developers write safer code by detecting certain common mistakes. It is enabled by adding "use strict"; at the beginning of a script or function.

Important Points:

1. Prevents Undeclared Variables: It does not allow assigning values to undeclared variables.
2. Avoids Duplicate Parameters: It prevents duplicate parameter names in functions.
3. Safer Coding: It converts certain silent errors into errors, making problems easier to identify.
4. Restricts Unsafe Operations: It prevents certain unsafe actions, such as deleting declared variables.
5. Improves Code Quality: It encourages developers to write more reliable and maintainable code.
6. Scope: It can be applied to an entire script or an individual function.

Example:

```
"use strict";
```

let age = 20;

console.log(age);

// Undeclared variable causes an error

// x = 10;

Output:

20

If x = 10; is uncommented, JavaScript throws a ReferenceError because x has not been declared.

Advantages:

- Helps detect programming mistakes.
- Prevents accidental global variables.
- Makes debugging easier.
- Encourages safer JavaScript code.

Q29. Explain the advantages of using jQuery over plain JavaScript.

Definition: jQuery is a JavaScript library that simplifies common web development tasks such as DOM manipulation, event handling, animations, and AJAX requests.

Important Points:

1. Simple Syntax: jQuery often performs tasks with fewer lines of code.
2. Easy DOM Manipulation: Methods such as text(), html(), and append() simplify content changes.
3. Easy Event Handling: The on() method makes it convenient to attach event handlers.
4. Built-in Effects: Methods such as hide(), show(), and fadeIn() provide simple animations.
5. AJAX Support: Methods such as $.get(), $.post(), and $.ajax() simplify server communication.
6. Cross-Browser Compatibility: jQuery handles many historical browser differences in supported operations.

Example:

Plain JavaScript:

```
document.getElementById("demo")
  .textContent = "Hello World";
```

Using jQuery:

```
$("#demo").text("Hello World");
```

Comparison Table:

Feature	Plain JavaScript	jQuery
Element selection	querySelector()	$()
Content change	textContent	text()
Event handling	addEventListener()	on()
Animations	Often requires custom code or Web Animations API	Built-in effects

AJAX	fetch() or other APIs	$.ajax(), $.get()

Q30. Explain the role of HTML5 APIs in modern web development.

Definition: HTML5 APIs are browser-provided interfaces that allow web applications to perform advanced operations such as storing data, drawing graphics, accessing location, and handling multimedia.

Important HTML5 APIs:

1. Geolocation API: Retrieves the user's geographical location after permission is granted.
2. Web Storage API: Stores data in the browser using localStorage and sessionStorage.
3. Canvas API: Draws graphics, shapes, and animations using JavaScript.
4. Drag and Drop API: Allows users to drag and drop elements or supported data.
5. Audio and Video: The <audio> and <video> elements support multimedia playback without requiring external plugins.
6. Web Workers API: Runs JavaScript tasks in background threads to help keep the user interface responsive.

Example: Web Storage API

<script>

localStorage.setItem("username", "Rishabh");

let name = localStorage.getItem("username");

console.log(name);

</script>

Output: Rishabh

Comparison of Common APIs:

API	**Purpose**
Geolocation	Accesses location with permission
Web Storage	Stores data in the browser
Canvas	Draws graphics
Drag and Drop	Supports dragging and dropping
Audio/Video	Plays multimedia
Web Workers	Runs tasks in background threads

Role in Modern Web Development:

- Creates interactive and responsive web applications.
- Reduces the need for external plugins.
- Improves user experience through graphics and multimedia.
- Enables browser-based storage and background processing.
- Provides access to useful device features with appropriate permissions.

Full-stack

__/__/____

1> Explain React.js and it's main features. How is React diff from traditional javascript-based web development?

-> React is a javascript library used to create interactive and dynamic user interfaces (UI) for web applications. It was developed by Facebook (Meta). React uses reusable components to build the user interface efficiently.

=> Main features ->

1> Component-Based -> React divides the UI into small and reusable components such as Header, Navbar, Button and Footer.

2> Virtual Dom -> React uses virtual Dom to update only the required Part of the webPage, making the application faster.

3> JSX -> Allows writing HTML-like code inside javascript.

4> Reusable Components -> Components can be created once and used many times, reducing code.

5> Faster and Efficient -> React efficiently manages UI updates and Provides a smooth user experience.

React.js	Traditional javascript
-> uses components to build the UI.	usually works directly with HTML elements and Dom.
-> uses virtual Dom for faster updates.	Directly manipulates the Dom
-> supports JSX for writing UI code.	uses HTML and JavaScript seperately
-> suitable for large and complex application.	suitable for simple websites and small interations.

RISHABH 181

__/__/____

=> Example :→ function welcom() {
return <h1> Hello, Rishabh ! </h1>~
}

2) What is JSX in React ? Explain the syntax and advantages of JSX with suitable example. Also explain JSX vs normal HTML.

→ JSX (Javascript XML) is a syntax extension used in React.js that allows developers to write HTML-like code inside JavaScript.

→ JSX makes it easier to create and understand the user interface of React applications.

=> Syntax :→ const element = <h1> Hello world ! </h1>;

=> Advantages of JSX :→

1) Easy to understand :→ JSX uses HTML-like syntax, so it is easy to read.
2) Combines HTML & JS :→ Allows UI structure and Javascript logic together.
3) Supports JS Expressions :→ JS expressions can be written inside.
4) Improves Development :→ Makes creating React UI simple & organized.
5) Works with Components :→ Helps create and reuse React Components easily.

=> Example :→ function Welcome() {
Const name = "Rishabh";
return (

<h1> welcom, {name} ! </h1>
 this is a React component.

);
}

__/__/____

JSX	Normal HTML
→ Used in React	used to create normal web page
→ written inside Javascript	written as a separate HTML document.
→ uses classname instead of class	uses class
→ JS can be directly used with {}	Javascript is not directly written this way
→ usally one Parent element is returned	HTML does not have this React restriction

3) Explain ~~React~~ the lifecycle of React Component.

→ Component lifecycle means the different stages a Component goes through during its life.

1) Mounting :→ Mounting means the Component is created and displayed on the webpage for the first time.

methods :→ render(), ComponentDidMount()

2) Updating :→ Updating means to Component changes because it's state or Props change.

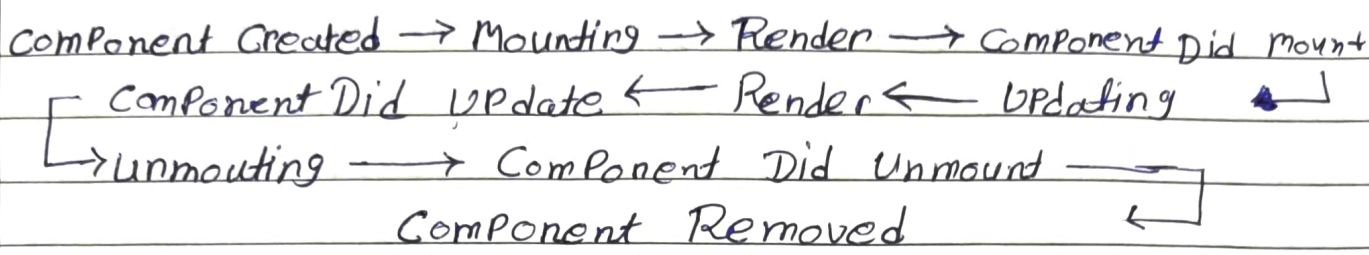
methods :→ render(), ComponentDidUpdate()

3) Unmounting :→ Unmounting means the Component is removed from the webPage.

method :→ ComponentDidUnmount()

__/__/____

=> Lifecycle :->



4) Explain Redux and Redux Saga. Why is Redux used for state management and how does Redux Saga handle asynchronous operations?

-> Redux is library used to manage the state of a React application in one central place.

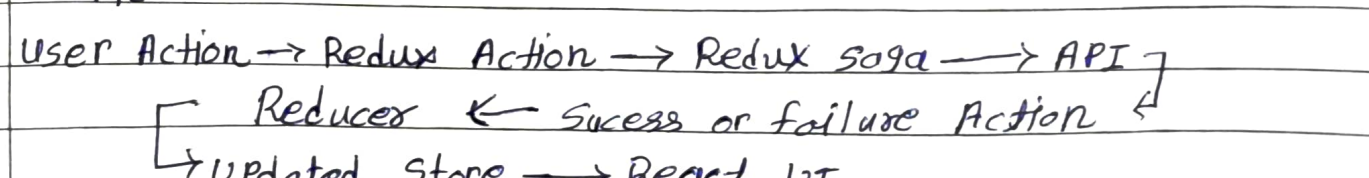
=> For example, if many Components need the same data, Redux can store that data in one place.

=> Used :-> It stores application state in one central place.
:-> Different Components can access the same state.
:-> It makes state management easier in large applications.
:-> Changes to state are handled in an organized way.

=> Redux Saga

-> Redux saga is middleware used with Redux to handle asynchronous operations such as API calls, data fetching, delays and other side effects.

=> work :->



RISHABH 181

__/__/____

5) How does Redux handle asynchronous function?

→ Redux manages application state. for asynchronous tasks lik API calls, Redux can use Redux Saga middleware.

=> work :→

I) user Action :→ user performs an action, like clicking a botn.
II) Redux :→ Redux receives the action.
III) Redux saga :→ Saga handles the asynchronous operation.
IV) API call :→ Saga calls the server.
V) Result :→ Saga waits for the API response.
VI) Redux update :→ Saga sends the result to Redux.
VII) state update :→ Redux updates the application state.

=> Flow :→

user Action → Redux → Redux saga → API call → Result
State updated ← Redux ←

=> Example :→

```
function* getData() {
    const data = yield call(api);
    yield put({type: "success", data});
}
```

6) Explain React ~~Routing~~ Router. How does routing work in React application?

→ React Router is a library used to create diffrent pages or views in React application.

Ex :=> a website can have :

- Home
- About
- Contact

→ React Router allows the users to move between these Pages.

RISHABH 191

__/__/____

=> How does Routing work:

User clicks a link
↓
React Router checks the URL
↓
Finds the matching route
↓
Display the required component

Ex:-> import {BrowserRouter, Routes, Route} from "react-router-dom";

```
function App() {
    return (
        <BrowserRouter>
        <Routes>
  <Route path="/" element={<h1>Home</h1>} />
  <Route path="/about" element {<h1>About</h1>} />
        </Routes>
    </BrowserRouter>
    );
}
```

7) Explain React Hooks. Discuss the purpose and usage of useState(), useEffect() and useContext() with example.

-> Hooks are special functions in React that allow functional components to use React features like state and other features.

=> There important Hooks are:

- useState() -> manages data
- useEffect() -> Performs actions after rendering
- useContext() -> gets shared data from context

RISHABH 181

__/__/____

(I) useState() :→ useState() is used to store and change data in a Component.

Example :→ Const [count, setcount] = useState(0);

(II) useEffect() :→ useEffect() is used to perform and action after the Component renders.

Example :→ useEffect(() => {
Console.log("Page loaded");
}, []);

(III) useContext() :→ useContext() is used to get shared data without passing Props through every Component.

Example :→ Const name = UseContext(nameContext);

8) What is Node.JS?

→ Node.js is a javascript runtime that allows us to run Javascript outside the web browser, mainly on the server.

Ex:→ We can use Node.js to create a web server.

1) Javascript Runtime :→ Node.js allows javascript to run on the server.

2) Event-Driven :→ Node.js responds to events such as user requests.

3) Non-Blocking :→ It can handle multiple requests without waiting for each task to finish.

4) NPM :→ NPM is used to install and manage Node.js Packages.

RISHABH 181

__/__/____

5) REPL :- R -> Read { E -> Evalute { P -> Print } L -> loop

It allows us to run js commands directly in termind

=> simple Architecture :- Client
↳ Node.js
↳ Event Loop
↳ Response.

9) How to create a simple server using Node.js with Express ?

-> Express.js is a framework for Node.js used to create web servers and web applications easily.

=> To create a simple Server:

1) install Express.
2) import Express into to program.
3) Create an express application.
4) Define a route to handle client requests.
5) Start the server using a Port numbers.

=> Code :-
```
Const express = require ("express");
    Const app = express();
        app.get("/", (req, res) => {
            res.send("Hello world");
        });
        app.listen(3000);
```
Run :- npm install express
node server.js
open :- http://localhost:3000
output :- Hello world

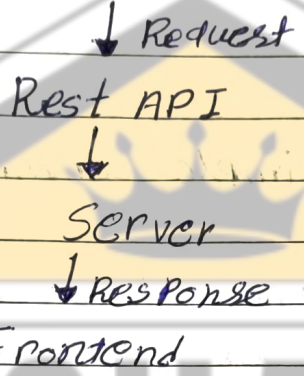
RISHABH 181

__/__/____

10) What are Restful APIs ? Explain HTTP methods such as GET, POST, PUT, PATch and Delete with suitable examples -Also explain the basic of authentication in Rest API.

-> Restful APIs is a way ~~of~~ for applications to Communicate with each other over the internet using HTTP.

=> For example :-> Fronted



=> HTTP methods :-> HTTPS methods tell the server what operation we want to Perform.

GET :->used to retrieve data
Post :-> used to create new data
Put :->used to completely update existing data.
Patch :-> used to update only some Part of existing data.
Delete :-> used to remove data.

=> Authentication in REST APIs

-> Authentication means checking whether the user is allowed to access the API

-> for example :-> Login -> server check user -> Authentication
Token Sent with API <- Token given <- successful

-> A Comon method is using a token or JWT.

RISHABH 181

__/__/____

11) What are Asynchronous and Synchronous operations ?

=> Synchronus :- Synchronus means tasks are performed one after another.

-> The Program waits for the first task to finish before starting the next task.

=> Example :-
```
Console.log ("start");
Console.log ("Task 1");
Console.log ("End");
```

Output :-
start
Task 1
End

Flow

Task 1
↳ Finish
↳ Task 2
↳ Finish
↳ Task 3

=> Asynchronus :- Asynchronus means the Program does not have to wait for a task to finish.

-> It can start another task while the first task is still running.

=> Example :-
```
Console.log ("start");
setTimeout (() => {
    Console.log ("Task 1");
}, 2000);
Console.log ("End");
```

output :- start
End
Task 1

Flow

Start task 1
↓
Don't wait
↓
Do task 2
↓
Task 1 finish

__/__/____

12> Compare MySQL and MongoDB:

MySQL	MongoDB
→ It is a relational Database	It is non relational Database.
→ Data stored in table	Data stored in documents and Collections.
→ uses SQL for queries.	Uses JSON-like documents.
→ Has a fixed/structured scheme.	Has a flexible scheme.
→ Relationship are commonly managed using JOINS.	Related data can be embedded in documents.
→ Suitable for highly structured data.	Suitable for flexible and changing data.
→ Example :- Banking, accounting system.	Example :- Real-time apps, content management.
→ Example :- SELect * from students;	Example :- { name : "Rishabh", age : 21 }

13> What is Data migration ? Explain the Process of migration data from MySQL to MongoDB. Discuss the challenges involved.

→ Data migration means moving data from one database to another database.

RISHABH 101

__/__/____

→ For example, moving data from MySQL to MongoDB is called data migration.

→ Process of MySQL to MongoDB migration

MySQL → Read data → Convert data → Create MongoDB documents → Insert data into MongoDB → check data

S:→1 :→ Read data from MySQL
S:→2 :→ Convert MySQL row and columns into MongoDB's document format.
S:→3 :→ Arrange the converted data as MongoDB documents.
S:→4 :→ store the documents in MongoDB collections.
S:→5 :→ check whether the data was transferred correctly

⇒ Challenges :→ Different data structures
:→ Relationships
:→ Data types
:→ Large amount of data
:→ Data loss/error

14) Explain how database are integrated with Express.js application. Describe the flow from an HTTP request to the database and back to the client.

→ Database integration means connecting an Express.js application with a database so that the application can store and retrieve data.

→ For example, an Express.js application can connect to MySQL or MongoDB.

RISHABH 181

__/__/____

=> Request - Response Flow

Client / Browser
↳ HTTP Request
↳ Express.js Route
↳ Database Query
↳ Database
↳ Result
↳ Express.js
↳ HTTP Response
↳ Client / Browser

S:→1> The browser sends an HTTP request to the Express.js server.
S:→2> Express checks the URL and finds the appropriate route.
S:→3> The route sends a query to the database to get, add, update.
S:→4> The database process the query and sends the result back to Express.
S:→5> Express sends the result back to the client as an HTTP response.

Example:→

```
app.get("/students", async (req, res) => {
    Const data = await db.find();
    res.json(data);
});
```

15> How do you Connect a Node.js application with mongoDB?
→ MongoDB is a NoSQL database used to store data in documents.

__/__/____

→ A node.js application can connect to MongoDB using the MongoDB driver.

S :→ 1> Install MongoDB Package
2> Import the mongoDB Package
3> create a MongoDB client.
4> Connect to MongoDB
5> select the Database
6> Perform database operations.

⇒ Code :→
```
Const {Mongoclient} = require ("MongoDB");
Const client = new Mongoclient ("mongodb://
                                 localhost:27017");

async function Connect () {
    await client.connect ();
    Const db = client.db ("school");
    Console.log ("MongoDB Connected");
}
Connect ();
```

OutPut :→ MongoDB Connected.

RISHABH 181